Lab 6 — Create a Web App with Azure App Service

In this lab you create a fully managed web app on Azure App Service, deploy a simple HTML page to it with a single CLI command, and explore the scaling options that PaaS gives you without touching a single server. This maps to the AZ-900 skill area **Describe Azure architecture and services** — Platform as a Service (PaaS) compute.

Run on https://portal.azure.com — sign in with your Azure free account (https://azure.microsoft.com/free).

Step 1 — Create the resource group (if it doesn't exist)

1. In the portal search bar, type **Resource groups** and open it.
2. Click **+ Create** and fill in:

- Subscription: your free subscription
- Resource group: `rg-az900-labs`
- Region: `Southeast Asia`

1. Click **Review + create** > **Create**. (Skip this step if `rg-az900-labs` already exists from an earlier lab.)

All labs in this course share this resource group so cleanup stays simple.

Step 2 — Create the Web App

1. Click **Create a resource** (top-left menu) > search **Web App** > **Create** > **Web App**.
2. On the **Basics** tab enter:

Field	Value
Subscription	your free subscription
Resource group	`rg-az900-labs`
Name	`app-az900-<yourinitials><random>` (e.g. `app-az900-tan4821` — must be globally unique)
Publish	`Code`
Runtime stack	`Node 20 LTS` (or `PHP 8.2`)
Operating System	`Linux`
Region	`Southeast Asia`

1. Under **Pricing plans**, click **Create new** for the Linux Plan, name it `plan-az900-free`, then click **Explore pricing plans** and select **Free F1** > **Select**.

2. Click **Review + create** > **Create** and wait for deployment (about 1 minute).

The name becomes part of a public URL (`<name>.azurewebsites.net`), which is why it must be globally unique.

Step 3 — Browse the default placeholder page

1. When deployment completes, click **Go to resource**.
2. On the **Overview** blade, find **Default domain** and click the `https://app-az900-....azurewebsites.net` link.
3. A default "Your web app is running and waiting for your content" page appears.

Notice you never created a VM, installed a web server, or opened a firewall port — App Service manages all of that for you.

Step 4 — Open Cloud Shell and create a simple page

1. Click the **Cloud Shell** icon (>_) in the top toolbar of the portal and choose **Bash** (create storage if prompted, accept defaults).
2. Create a folder and an `index.html`:

```
mkdir site && cd site
code index.html
```

The `code` command opens the built-in Cloud Shell editor. Paste this content:

```
<h1>Hello from Azure App Service!</h1>
<p>Deployed by <your name> — AZ-900 Lab 6</p>
```

1. Save with **Ctrl+S** and close the editor with **Ctrl+Q**.

Step 5 — Zip and deploy the page

```
zip site.zip index.html
```

This packages your file — App Service accepts zip packages as a deployment source.

```
az webapp deploy --resource-group rg-az900-labs --name
app-az900-<yourinitials><random> --src-path site.zip --type zip
```

`--type zip` tells the deployment engine to extract the archive into the web root (`/home/site/wwwroot`). Wait for the JSON output showing `"provisioningState": "Succeeded"`.

Step 6 — Verify the deployment

1. Go back to the browser tab with your web app and refresh (Ctrl+F5 to bypass cache).
2. Your "Hello from Azure App Service!" page replaces the placeholder.

One command took your code from Cloud Shell to a live, HTTPS-enabled public website — this is the core PaaS value proposition.

Step 7 — Explore Scale up (App Service plan)

1. On your web app's left menu, under **Settings**, click **Scale up (App Service plan)**.
2. Browse the tiers: **Dev/Test** (F1, B1), **Production** (P0v3, P1v3...). Note what higher tiers add: custom domains + TLS, staging slots, autoscale, more CPU/RAM.
3. **Do NOT change the tier** — stay on F1. Close the blade without applying.

Scaling **up** means moving to a bigger instance size (vertical scaling).

Step 8 — Explore Scale out (App Service plan)

1. Under **Settings**, click **Scale out (App Service plan)**.
2. Observe that on the Free F1 tier the instance count is fixed at 1, and rules-based/automatic scaling requires a paid tier.
3. **Do NOT scale** — just note that on Standard and above you could set autoscale rules (e.g. add an instance when CPU > 70%).

Scaling **out** means adding more instances of the same size (horizontal scaling) — the preferred cloud pattern for handling variable load.

Clean up

Keep the resource group `rg-az900-labs` — it is reused until Lab 16. Delete only this lab's resources:

```
az webapp delete --resource-group rg-az900-labs --name
app-az900-<yourinitials><random>
az appservice plan delete --resource-group rg-az900-labs --name plan-az900-free
--yes
```

Or in the portal: open `rg-az900-labs`, tick the web app and the App Service plan, click **Delete** and confirm. (F1 is free, but deleting keeps the resource group tidy.)

What you learned

- App Service is **PaaS**: Azure manages the OS, web server, patching, and TLS — you manage only your code (shared responsibility model).
- An **App Service plan** defines the compute (tier, size, region) that one or more web apps run on.
- Zip deploy via `az webapp deploy` is a simple, repeatable way to publish code to App Service.
- **Scale up** = bigger instances (vertical); **scale out** = more instances (horizontal); higher tiers unlock autoscale.
- App names must be globally unique because they form part of the public `*.azurewebsites.net` DNS name.

Files in this lab folder

File	What it is
`index.html`	The sample page to deploy — a styled version of the HTML in Step 4.
`deploy-webapp.sh`	Zips `index.html` and deploys it to your web app: `bash deploy-webapp.sh <app-name>`.

The scripts are optional conveniences — the lab steps above teach the same thing command by command. Upload a script to Cloud Shell (**Manage files > Upload**), or clone this repo there, then run it with `bash <script-name>`.